

Servicio: gestion_citas.py

Archivo: `services/gestion_citas.py` · Documento técnico-pedagógico · Generado 2026-05-20 13:46

Maneja el registro, edición y eliminación de citas. Es la 'memoria' del sistema sobre quién tiene cita, cuándo y para qué examen.

Resumen general

Este archivo se conservó en gran medida intacto: las funciones originales (*generar_id_cita*, *registrar_cita*, *validar_tope_atenciones*) siguen ahí. Lo que añadimos son **tres ayudantes** que la interfaz gráfica necesita para mostrar visualmente la disponibilidad de horarios.

ANTES	AHORA
No había lista de horarios estandarizada (el usuario escribía cualquier hora)	<code>HORAS_DISPONIBLES = ['19:00', '19:15', '19:30', ... '22:00']</code> La UI muestra estos horarios como grilla.
No se podía saber rápidamente cuántas citas hay a cierta hora	<code>contar_citas_horario(fecha, hora)</code> Devuelve el conteo instantáneo.
No se podía saber el total de citas de un día	<code>contar_citas_fecha(fecha)</code> Usado en el combo de fechas para indicar carga.

1. La lista HORAS_DISPONIBLES

El sistema piloto opera entre 19:00 y 22:00 con franjas de 15 minutos. Esta lista contiene exactamente esas franjas. La **grilla visual** del formulario de citas recorre esta lista para dibujar las celdas coloreadas.

```
HORAS_DISPONIBLES = [  
    "19:00", "19:15", "19:30", "19:45",  
    "20:00", "20:15", "20:30", "20:45",  
    "21:00", "21:15", "21:30", "21:45",  
    "22:00",  
]
```

Para principiantes: Las **constantes** (variables que no cambian) se escriben en MAYÚSCULAS por convención en Python. Así cualquier programador entiende rápido que ese valor está fijo y se usa como referencia en varios sitios.

2. Las funciones de conteo

Son funciones muy cortas pero importantes: la UI las llama cada vez que redibuja la grilla de horarios o el combo de fechas, para mostrar al usuario cuán llena está cada hora del día.

```
def contar_citas_horario(fecha, hora):  
    return sum(1 for c in lista_citas  
               if c.fecha == fecha and c.hora == hora)  
  
def contar_citas_fecha(fecha):  
    return sum(1 for c in lista_citas if c.fecha == fecha)
```

Para principiantes: La expresión `sum(1 for c in lista if condición)` es una manera elegante de Python para contar cuántos elementos cumplen una condición. Es equivalente a un bucle `for` tradicional pero ocupa una sola línea.

3. Por qué se conservaron las funciones CLI

Las funciones `editar_cita()` y `eliminar_cita()` originales seguían usando `input()` de consola. Las dejamos intactas porque el sistema sigue ofreciendo un modo CLI (`python main.py cli`) por si alguna vez se necesita un acceso sin entorno gráfico. La interfaz gráfica no las usa: implementa la edición y eliminación con sus propios diálogos modales.

Para principiantes: Esto se llama '**compatibilidad hacia atrás**': añadir funcionalidad nueva sin romper la antigua. Es valioso cuando varios sistemas dependen del mismo archivo.